

Analytical Steiner Derivatives

Closed-form first and second derivatives of triple-point contributions

May 2026

Abstract

At a triple point, three partition boundaries meet at a Fermat–Torricelli (Steiner) point **S**. The Phase 2 perimeter-refinement optimizer needs the first and second derivatives of the Steiner perimeter and area contributions with respect to the variable-point parameters λ . This document derives all of them in closed form by applying the implicit-function theorem to the Fermat-point optimality condition. Every quantity here is implemented analytically in `src/partition/vectorized_steiner.py`; the companion document `docs/math/01-phase2-derivatives` covers the regular (non-Steiner) perimeter and area derivatives, and the finite-difference schemes that these closed forms replaced.

Contents

1	Overview	2
2	Notation and Setup	2
3	The Fermat Point as an Implicit Equation	2
4	First Derivative of the Steiner Point	3
5	First Derivatives of Perimeter and Area	3
5.1	Steiner perimeter gradient	3
5.2	Steiner area Jacobian	4
6	Second Derivative of the Steiner Point	4
7	Chain Rule to λ-Space	5
8	Steiner Hessians	5
8.1	Perimeter Hessian	5
8.2	Area Hessian	5
9	Degenerate Case	5

1 Overview

A *triple point* arises at a mesh triangle where three partition cells meet. Inside that triangle the three cell boundaries do not run straight to a shared mesh vertex; instead they meet at the *Fermat–Torricelli point* $\mathbf{S}$ — the point minimising the total distance to the triangle’s three variable points [1]. The Steiner construction contributes to both the perimeter objective and the per-cell area constraints, so the optimizer needs

$$\frac{\partial P_{\text{st}}}{\partial \lambda_i}, \quad \frac{\partial A_k^{\text{st}}}{\partial \lambda_i}, \quad \frac{\partial^2 P_{\text{st}}}{\partial \lambda_i \partial \lambda_j}, \quad \frac{\partial^2 A_k^{\text{st}}}{\partial \lambda_i \partial \lambda_j}. \quad (1)$$

The forward value $\mathbf{S}$ is a closed-form barycentric expression, but its derivatives are not elementary: $\mathbf{S}$ is defined only implicitly, as the solution of an optimality condition. The key is the implicit-function theorem. Once $\partial \mathbf{S} / \partial \lambda_i$ and $\partial^2 \mathbf{S} / \partial \lambda_i \partial \lambda_j$ are known, the perimeter and area derivatives follow by the chain rule.

Sections 2–4 derive $\partial \mathbf{S} / \partial p_k$; Section 5 gives the first derivatives of the perimeter and area; Sections 6–7 derive $\partial^2 \mathbf{S} / \partial p_k \partial p_l$ and lift it to λ -space; Section 8 assembles the perimeter and area Hessians; Section 9 treats the degenerate case.

2 Notation and Setup

At one triple point three variable points $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3 \in \mathbb{R}^3$ each depend affinely on a scalar parameter:

$$\mathbf{p}_i(\lambda_i) = \lambda_i \mathbf{V}[v_{1,i}] + (1 - \lambda_i) \mathbf{V}[v_{2,i}], \quad \mathbf{d}_i := \frac{\partial \mathbf{p}_i}{\partial \lambda_i} = \mathbf{V}[v_{1,i}] - \mathbf{V}[v_{2,i}]. \quad (2)$$

The edge direction $\mathbf{d}_i$ is constant and $\partial^2 \mathbf{p}_i / \partial \lambda_i^2 = \mathbf{0}$: every variable point is *linear* in its own parameter. This is the single structural fact that makes the chain rule below tractable.

Let $\mathbf{S}$ be the Fermat–Torricelli point of the triangle $(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3)$. Define, for $i = 1, 2, 3$,

$$\mathbf{u}_i := \mathbf{p}_i - \mathbf{S}, \quad r_i := \|\mathbf{u}_i\|, \quad \mathbf{n}_i := \mathbf{u}_i / r_i, \quad (3)$$

$$K_i := \frac{(\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^\top)}{r_i}, \quad M := K_1 + K_2 + K_3. \quad (4)$$

Each K_i is symmetric positive semidefinite; M is symmetric positive definite whenever the triangle is non-degenerate (Section 9).

Code: triple-point geometry

`src/partition/vectorized_steiner.py`: `_compute_tp_geometry(pa)` returns $(\mathbf{p}, \mathbf{S}, r, \mathbf{n}, K, M)$ per triple point; `_compute_tp_steiner(pa)` is the shared forward-value / degeneracy core.

3 The Fermat Point as an Implicit Equation

$\mathbf{S}$ is the unique minimiser of the strictly convex function

$$G(\mathbf{S}) = \|\mathbf{p}_1 - \mathbf{S}\| + \|\mathbf{p}_2 - \mathbf{S}\| + \|\mathbf{p}_3 - \mathbf{S}\|. \quad (5)$$

Since $\partial \|\mathbf{p}_i - \mathbf{S}\| / \partial \mathbf{S} = -\mathbf{n}_i$, the first-order optimality condition $\nabla_{\mathbf{S}} G = \mathbf{0}$ is

$$F(\mathbf{S}, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) := \sum_{i=1}^3 \mathbf{n}_i = \mathbf{0}. \quad (6)$$

Geometrically the three unit vectors from $\mathbf{S}$ to the variable points sum to zero — equivalently they make 120° angles, the defining property of the Fermat point when every triangle angle is below 120° . Equation (6) defines $\mathbf{S}$ implicitly; differentiating it is the route to all the derivatives below.

Proposition 3.1 (Derivative of a unit vector). *If $\mathbf{u}$ depends on a variable $\mathbf{x}$ with constant Jacobian $\partial\mathbf{u}/\partial\mathbf{x} = A$, then the unit vector $\mathbf{n} = \mathbf{u}/r$, $r = \|\mathbf{u}\|$, satisfies*

$$\frac{\partial\mathbf{n}}{\partial\mathbf{x}} = \frac{(\mathbf{I} - \mathbf{n}\mathbf{n}^\top)}{r} A = K A, \quad \frac{\partial r}{\partial\mathbf{x}} = \mathbf{n}^\top A. \quad (7)$$

Proof. $\partial r/\partial\mathbf{x} = (\mathbf{u}^\top/r) A = \mathbf{n}^\top A$. Then $\partial\mathbf{n}/\partial\mathbf{x} = A/r - (\mathbf{u}/r^2) \partial r/\partial\mathbf{x} = (\mathbf{I} - \mathbf{n}\mathbf{n}^\top)A/r = KA$. $\square$

4 First Derivative of the Steiner Point

Differentiate the optimality condition (6) with respect to $\mathbf{p}_k$. By the implicit-function theorem,

$$\frac{\partial F}{\partial\mathbf{S}} \frac{\partial\mathbf{S}}{\partial\mathbf{p}_k} + \frac{\partial F}{\partial\mathbf{p}_k} = \mathbf{0}. \quad (8)$$

Using Proposition 3.1 with $\mathbf{u}_i = \mathbf{p}_i - \mathbf{S}$: varying $\mathbf{S}$ gives $\partial\mathbf{u}_i/\partial\mathbf{S} = -\mathbf{I}$, so $\partial F/\partial\mathbf{S} = \sum_i K_i(-\mathbf{I}) = -M$; varying $\mathbf{p}_k$ gives $\partial\mathbf{u}_i/\partial\mathbf{p}_k = \delta_{ik}\mathbf{I}$, so $\partial F/\partial\mathbf{p}_k = K_k$. Hence

$$-M \frac{\partial\mathbf{S}}{\partial\mathbf{p}_k} + K_k = \mathbf{0} \implies \boxed{\frac{\partial\mathbf{S}}{\partial\mathbf{p}_k} = M^{-1}K_k} \quad (9)$$

Remark 4.1 (Translation invariance). Summing (9) over k gives $\sum_k \partial\mathbf{S}/\partial\mathbf{p}_k = M^{-1}(K_1 + K_2 + K_3) = M^{-1}M = \mathbf{I}$. Translating all three vertices by a common vector translates $\mathbf{S}$ identically. The implementation asserts this identity as a self-check.

The implementation never forms M^{-1} explicitly: it solves $M X = K_k$ with `np.linalg.solve`, which is better conditioned near the degenerate case and is natively batched over all triple points.

Code: first derivative of S

```
src/partition/vectorized_steiner.py:      _compute_dS_dp(geom)      returns      dS/dp_k;
_compute_dS_dlambda(geom, dS_dp) lifts it to  $\lambda$ -space via  $\partial\mathbf{S}/\partial\lambda_k = (\partial\mathbf{S}/\partial\mathbf{p}_k) \mathbf{d}_k$ .
```

5 First Derivatives of Perimeter and Area

5.1 Steiner perimeter gradient

Per cell per triple point the Steiner perimeter contribution is $\ell_a + \ell_b - \ell_{ab}$, with

$$\ell_a = \|\mathbf{p}_a - \mathbf{S}\|, \quad \ell_b = \|\mathbf{p}_b - \mathbf{S}\|, \quad \ell_{ab} = \|\mathbf{p}_a - \mathbf{p}_b\|, \quad (10)$$

where $\mathbf{p}_a, \mathbf{p}_b$ are the two variable points the cell borders. Writing $\mathbf{n}_a = (\mathbf{p}_a - \mathbf{S})/\ell_a$ and using $\partial\mathbf{p}_a/\partial\lambda_i = \delta_{ai}\mathbf{d}_i$ and $\partial\mathbf{S}/\partial\lambda_i = (M^{-1}K_i)\mathbf{d}_i$,

$$\frac{\partial\ell_a}{\partial\lambda_i} = \mathbf{n}_a^\top (\delta_{ai}\mathbf{d}_i - (M^{-1}K_i)\mathbf{d}_i). \quad (11)$$

The chord term ℓ_{ab} does not involve $\mathbf{S}$; its gradient is the regular perimeter-segment gradient of `docs/math/01-phase2-derivatives`.

5.2 Steiner area Jacobian

The Steiner area of a cell at a triple point is $A^{\text{st}} = \frac{1}{2} \|(\mathbf{p}_b - \mathbf{p}_a) \times (\mathbf{S} - \mathbf{p}_a)\| + \text{corner}$. The corner triangle has a fixed mesh vertex and reuses the regular 1-inside area-Jacobian formula. The *void* triangle has all three vertices moving, so write $\mathbf{e}_1 = \mathbf{p}_b - \mathbf{p}_a$, $\mathbf{e}_2 = \mathbf{S} - \mathbf{p}_a$, $\mathbf{C} = \mathbf{e}_1 \times \mathbf{e}_2$, $\hat{\mathbf{n}} = \mathbf{C} / \|\mathbf{C}\|$. Then

$$\frac{\partial A_{\text{void}}^{\text{st}}}{\partial \lambda_i} = \frac{1}{2} \hat{\mathbf{n}}^\top \left(\frac{\partial \mathbf{e}_1}{\partial \lambda_i} \times \mathbf{e}_2 + \mathbf{e}_1 \times \frac{\partial \mathbf{e}_2}{\partial \lambda_i} \right), \quad (12)$$

the generalised scalar-triple-product derivative; the per- λ velocities $\partial \mathbf{e}_1 / \partial \lambda_i$, $\partial \mathbf{e}_2 / \partial \lambda_i$ are built from $\mathbf{d}_i$ and $\partial \mathbf{S} / \partial \lambda_i$.

Code: Steiner first derivatives

```
src/partition/vectorized_steiner.py: compute_steiner_perimeter_gradient_analytical(pa),
compute_steiner_area_jacobian_analytical(pa) and its _sparse twin.
```

6 Second Derivative of the Steiner Point

Differentiate (9) with respect to $\mathbf{p}_l$. Using $\partial(M^{-1})/\partial \mathbf{p}_l = -M^{-1}(\partial M/\partial \mathbf{p}_l)M^{-1}$,

$$\frac{\partial^2 \mathbf{S}}{\partial \mathbf{p}_k \partial \mathbf{p}_l} = -M^{-1} \frac{\partial M}{\partial \mathbf{p}_l} M^{-1} K_k + M^{-1} \frac{\partial K_k}{\partial \mathbf{p}_l}. \quad (13)$$

The two remaining ingredients are $\partial K_j / \partial \mathbf{p}_l$ and $\partial M / \partial \mathbf{p}_l = \sum_j \partial K_j / \partial \mathbf{p}_l$.

Define the rank-two tensor

$$T_{jl} := \frac{\partial \mathbf{u}_j}{\partial \mathbf{p}_l} = \delta_{jl} \mathbf{I} - \frac{\partial \mathbf{S}}{\partial \mathbf{p}_l}, \quad (14)$$

which carries both the explicit dependence ($\mathbf{p}_j$ when $l = j$) and the implicit dependence (through $\mathbf{S}$, always). By Proposition 3.1,

$$\frac{\partial r_j}{\partial \mathbf{p}_l} = \mathbf{n}_j^\top T_{jl}, \quad \frac{\partial \mathbf{n}_j}{\partial \mathbf{p}_l} = K_j T_{jl}. \quad (15)$$

Differentiating $K_j = (\mathbf{I} - \mathbf{n}_j \mathbf{n}_j^\top) / r_j$ by the quotient rule,

$$\frac{\partial K_j}{\partial \mathbf{p}_l} = -\frac{1}{r_j} \left[(K_j T_{jl}) \mathbf{n}_j^\top + \mathbf{n}_j (K_j T_{jl})^\top + K_j \frac{\partial r_j}{\partial \mathbf{p}_l} \right]. \quad (16)$$

Remark 6.1 (Tensor structure of (16)). $\partial K_j / \partial \mathbf{p}_l$ is a rank-three object: two indices from the 3×3 matrix K_j and one from the differentiation direction $\mathbf{p}_l$. The three terms of (16) are *outer products* over the matrix and derivative slots, *not* matrix products — $(K_j T_{jl}) \mathbf{n}_j^\top$ means the entry $[a, b, c] = (K_j T_{jl})_{ac} (\mathbf{n}_j)_b$. The implementation builds each term with an explicit `np.einsum` index string; a matrix product would silently contract the wrong axis.

Substituting (16) (summed over j for $\partial M / \partial \mathbf{p}_l$) into (13) gives $\partial^2 \mathbf{S} / \partial \mathbf{p}_k \partial \mathbf{p}_l$, a rank-three tensor per index pair (k, l) .

Remark 6.2 (Second-order translation invariance). Differentiating Remark 4.1's identity $\sum_k \partial \mathbf{S} / \partial \mathbf{p}_k = \mathbf{I}$ with respect to $\mathbf{p}_l$ gives $\sum_k \partial^2 \mathbf{S} / \partial \mathbf{p}_k \partial \mathbf{p}_l = \mathbf{0}$ for every l . The implementation asserts this, together with the mixed-partial symmetry $\partial^2 \mathbf{S} / \partial \mathbf{p}_k \partial \mathbf{p}_l = \partial^2 \mathbf{S} / \partial \mathbf{p}_l \partial \mathbf{p}_k$ (with the corresponding swap of the two input-component axes), as self-checks.

7 Chain Rule to λ -Space

Because each $\mathbf{p}_k$ is affine in its own λ_k with constant velocity $\mathbf{d}_k$ and zero acceleration, the chain rule has no second-order vertex term:

$$\frac{\partial^2 \mathbf{S}}{\partial \lambda_i \partial \lambda_j} = \left(\frac{\partial^2 \mathbf{S}}{\partial \mathbf{p}_i \partial \mathbf{p}_j} \right) \cdot \mathbf{d}_i \cdot \mathbf{d}_j, \quad (17)$$

contracting the two input-component axes of the rank-three tensor with the edge directions.

Code: second derivative of S

```
src/partition/vectorized_steiner.py: _compute_d2S_dp2(geom, dS_dp) builds  $\partial^2 \mathbf{S} / \partial \mathbf{p}_k \partial \mathbf{p}_l$ 
from (13)–(16); _compute_d2S_dlambda(geom, d2S_dp2) applies (17).
```

8 Steiner Hessians

8.1 Perimeter Hessian

For a leg $\ell_a = \|\mathbf{p}_a - \mathbf{S}\|$, write $\Delta_i = \partial(\mathbf{p}_a - \mathbf{S}) / \partial \lambda_i = \delta_{ai} \mathbf{d}_i - \partial \mathbf{S} / \partial \lambda_i$ and $\Delta_{ij} = -\partial^2 \mathbf{S} / \partial \lambda_i \partial \lambda_j$ (the variable point contributes no second-order term). Then

$$\frac{\partial^2 \ell_a}{\partial \lambda_i \partial \lambda_j} = \Delta_i^\top \frac{(\mathbf{I} - \mathbf{n}_a \mathbf{n}_a^\top)}{\ell_a} \Delta_j + \mathbf{n}_a^\top \Delta_{ij}. \quad (18)$$

The first term is the projected dot product familiar from the regular perimeter Hessian; the second term is the contribution of the curved trajectory of $\mathbf{S}$ and is the reason the second derivative (17) is needed — omitting it makes the Hessian wrong, not merely approximate. The chord ℓ_{ab} reuses the regular perimeter-segment Hessian. Each leg produces a full symmetric 3×3 block over the triple point’s three variable points.

8.2 Area Hessian

For the void triangle area $\frac{1}{2} \|\mathbf{C}\|$, $\mathbf{C} = \mathbf{e}_1 \times \mathbf{e}_2$, $\hat{\mathbf{n}} = \mathbf{C} / \|\mathbf{C}\|$,

$$\frac{\partial^2 A_{\text{void}}^{\text{st}}}{\partial \lambda_i \partial \lambda_j} = \frac{1}{2} \left[\left(\frac{\partial \mathbf{C}}{\partial \lambda_i} \right)^\top \frac{(\mathbf{I} - \hat{\mathbf{n}} \hat{\mathbf{n}}^\top)}{\|\mathbf{C}\|} \frac{\partial \mathbf{C}}{\partial \lambda_j} + \hat{\mathbf{n}}^\top \frac{\partial^2 \mathbf{C}}{\partial \lambda_i \partial \lambda_j} \right]. \quad (19)$$

The second-order cross-product term carries $\partial^2 \mathbf{e}_2 / \partial \lambda_i \partial \lambda_j = \partial^2 \mathbf{S} / \partial \lambda_i \partial \lambda_j$ — the area analogue of the perimeter’s $\mathbf{n}_a^\top \Delta_{ij}$ term. The corner triangle has no moving mesh vertex, so its $\partial^2 \mathbf{C}$ retains only the mixed first-derivative cross terms.

Code: Steiner Hessians

```
src/partition/vectorized_steiner.py: compute_steiner_perimeter_hessian_analytical(pa),
compute_steiner_area_hessian_analytical(pa, multipliers). The symmetric  $3 \times 3$  blocks
are scattered with np.add.at on the pre-computed offset array pa.tp_hess_off.
```

9 Degenerate Case

When some triangle angle reaches or exceeds 120° , the minimiser of G collapses onto the obtuse vertex: $\mathbf{S} = \mathbf{p}_{\text{obtuse}}$ identically. The optimality condition (6) no longer holds (three unit vectors cannot make 120° angles), and M is singular there — $r_{\text{obtuse}} \rightarrow 0$, so K_{obtuse} diverges. On the degenerate branch the derivatives are elementary:

$$\frac{\partial \mathbf{S}}{\partial \mathbf{p}_k} = \delta_{k, \text{obtuse}} \mathbf{I}, \quad \frac{\partial^2 \mathbf{S}}{\partial \mathbf{p}_k \partial \mathbf{p}_l} = \mathbf{0}. \quad (20)$$

Remark 9.1 (Non-smoothness). The two branches do not agree at the 120° boundary — $\mathbf{S}$ as a function of $\boldsymbol{\lambda}$ is genuinely non-smooth there. The boundary is a measure-zero set in parameter space, so the optimizer essentially never sits on it, but each evaluation must commit to one branch. The detection (`degen_mask`, `obtuse_idx`) lives in the shared core `_compute_tp_steiner`, so the forward value $\mathbf{S}$ and its derivatives are always evaluated on the same branch. The non-degenerate formulas mask out degenerate rows before inverting M .

Summary table

Quantity	Method	Implementing function
$\partial \mathbf{S} / \partial \mathbf{p}_k$	Analytical (9)	<code>_compute_dS_dp</code>
$\partial^2 \mathbf{S} / \partial \mathbf{p}_k \partial \mathbf{p}_l$	Analytical (13)–(16)	<code>_compute_d2S_dp2</code>
$\partial \mathbf{S} / \partial \lambda_i, \partial^2 \mathbf{S} / \partial \lambda_i \partial \lambda_j$	Chain rule (17)	<code>_compute_dS_dlambd</code> , <code>_compute_d2S_dlambd</code>
Steiner perimeter gradient	Analytical (11)	<code>compute_steiner_perimeter_gradient_analytical</code>
Steiner area Jacobian	Analytical (12)	<code>compute_steiner_area_jacobian_analytical</code>
Steiner perimeter Hessian	Analytical (18)	<code>compute_steiner_perimeter_hessian_analytical</code>
Steiner area Hessian	Analytical (19)	<code>compute_steiner_area_hessian_analytical</code>

The closed forms are validated against finite differences by `testing/test_steiner_gradient_analytical.py`, `testing/test_steiner_hessian_analytical.py`, and `testing/test_steiner_degenerate_case.py`, and the assembled Lagrangian Hessian by `testing/test_exact_hessian_vs_fd.py`.

References

- [1] Harold W. Kuhn. Steiner’s problem revisited. pages 52–70, 1974. Background reference for the Fermat–Torricelli construction and the barycentric-weight formula for the Steiner point.